```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Load and prepare data
iris = load_iris()
X = iris.data
y = iris.target.reshape(-1, 1)

# Use sparse_output instead of sparse
encoder = OneHotEncoder(sparse_output=False)
y = encoder.fit_transform(y)

scaler = StandardScaler()
X = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Neural Network with Backpropagation
class NeuralNetwork:
    def __init__(self, input_size, hidden_size, output_size, lr=0.1):
        self.W1 = np.random.randn(input_size, hidden_size)
        self.b1 = np.zeros((1, hidden_size))
        self.W2 = np.random.randn(hidden_size, output_size)
        self.b2 = np.zeros((1, output_size))
        self.lr = lr

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def sigmoid_derivative(self, z):
        return z * (1 - z)

    def forward(self, X):
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = self.sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = self.sigmoid(self.z2)
        return self.a2
```

```
    def backward(self, X, y, output):
        error = y - output
        d_output = error * self.sigmoid_derivative(output)
        error_hidden = d_output.dot(self.W2.T)
        d_hidden = error_hidden * self.sigmoid_derivative(self.a1)

        self.W2 += self.a1.T.dot(d_output) * self.lr
        self.b2 += np.sum(d_output, axis=0, keepdims=True) * self.lr
        self.W1 += X.T.dot(d_hidden) * self.lr
        self.b1 += np.sum(d_hidden, axis=0, keepdims=True) * self.lr

    def train(self, X, y, epochs=1000):
        for epoch in range(epochs):
            output = self.forward(X)
            self.backward(X, y, output)
            if epoch % 200 == 0:
                loss = np.mean(np.square(y - output))
                print(f"Epoch {epoch}, Loss: {loss:.4f}")

    def predict(self, X):
        output = self.forward(X)
        return np.argmax(output, axis=1)

# Run training and testing
nn = NeuralNetwork(input_size=4, hidden_size=5, output_size=3, lr=0.1)
nn.train(X_train, y_train, epochs=1000)

predictions = nn.predict(X_test)
true_labels = np.argmax(y_test, axis=1)
accuracy = np.mean(predictions == true_labels)
print("Test Accuracy:", accuracy)

# Classify a new sample
new_sample = np.array([[5.1, 3.5, 1.4, 0.2]])  # Example flower
new_sample = scaler.transform(new_sample)
print("Predicted Class:", iris.target_names[nn.predict(new_sample)][0])
```

```
Epoch 0, Loss: 0.3363
Epoch 200, Loss: 0.0118
Epoch 400, Loss: 0.0106
```

```
Epoch 600, Loss: 0.0103
Epoch 800, Loss: 0.0102
Test Accuracy: 1.0
Predicted Class: setosa
```